1 The Pre-Trip module
1.1 Architecture
1.2 System interactions
2 Design decisions
3 Data model
4 Status & lifecycle

SCREENS

5.1 Dashboard
5.2 Request / Traveller
5.3 Trip details
5.4 Estimates
5.5 Charging
5.6 Policy review
5.7 Review & submit
5.8 Approver / DOA
5.9 TE claim
5.10 Configuration

LOGIC

6 Validation rules
7 Approval routing
8 Class entitlement
9 Charging & allocation
10 Claim & auto-grant
11 Configuration ref.
12 TMC integration
13 Reference masters
14 Group & guest bookings
15 Enumerations
16 Open decisions

NTU EMPLOYEE CLAIM SYSTEM · MODULE BUILD PACK

ECS Pre-Trip Travel Request — Build Specification

A vendor-ready specification of the Pre-Trip Travel Request module: every screen, field, validation rule, program algorithm and configuration switch needed to build it, plus the design decisions that shaped them.

Audience Implementation vendor / OutSystems build team

Status Prototype-validated, build-ready

Companion Feature Walkthrough artifact (end-user view)

1 The Pre-Trip module

The Pre-Trip module captures a travel request before a trip, derives entitlements and the estimated cost, routes it for approval, issues a Travel Authorisation, hands a governed booking instruction to a Travel Management Company (TMC), and seeds the downstream Travel Expense (TE) claim.

Purpose

The module closes the gap between an employee's **intent to travel** and a **booked, policy-compliant, financially authorised trip**. It replaces email/spreadsheet approvals and retrospective booking with a single governed flow: capture the trip, derive the entitlement and cost, obtain the right financial authority (DOA), and instruct the TMC — **before any spend is committed**. It then pre-populates the post-trip expense claim so the claim reconciles to what was approved.

OUTCOME

Policy enforced before spend

Caps, class entitlement and exceptions are evaluated and routed to the right approver before booking, not discovered at claim time.

OUTCOME

One Travel Authorisation per trip

A dated, amount-bound TA (TA-YYYY-NNNNNN) is the single source of truth the TMC and the claim both reconcile against.

OUTCOME

Clean TMC hand-off & claim

A governed, field-filtered booking instruction per traveller; a pre-filled TE claim whose DOA is auto-granted when it matches the estimate.

Capabilities

Request capture

Individual, **group** (coordinator-raised, multi-traveller, with **Excel/CSV bulk upload**) and **guest** (non-employee) requests.

Itinerary & trip types

Round-trip / one-way / multi-city (multi-country), always-on leg builder, personal segments.

Entitlement derivation

Travel class derived from the itinerary (longest-duration leg), per traveller for groups.

Cost estimation

Airfare, accommodation (cap-checked), ODA (rate × eligible days), incidentals; FX conversion; sponsorship.

Policy engine

Common + pre-trip rules → pass / exception / hard stop; hard stops block submission.

Configurable approval routing

RO · Additional · Funding Owner · Cross-BA · Exception · DOA — computed from context + settings.

Charging & allocation

ECS cascade, per-line multi-account splits, auto-allocate by Workday department.

Authorisation & booking

TA issue, validity & booking-deadline tracking, booking-status lifecycle.

TMC hand-off

Governed payload (integration contract), group fan-out (one instruction per traveller), inbound status.

Claim seeding

Pre-populates the TE claim; DOA auto-granted within tolerance.

Amendments & audit

Material-amendment re-approval, immutable version snapshots, field-level audit.

Configuration

Runtime Module Settings drive routing switches, thresholds, caps basis and tolerances.

Scope & boundary

The module **extends the existing ECS / ECS-Travel platform**. It does not re-implement HR, finance or DOA masters — it **reads** them. The build team replaces the prototype's mock service layer with real calls to ECS/OutSystems.

REUSED (READ-ONLY)

ECS reference masters

Employees & appointments, reporting lines, DOA lines, charging accounts (CC/WBS), policy caps & ODA rates, FX rates, workflow amount bands, roles, TMC providers, location master.

NEW (TRANSACTIONAL)

Pre-Trip data store

Travel requests, travellers, itinerary legs, estimated expenses, charging allocations, policy checks, approval steps, bookings, TE claims, audit, attachments.

INTEGRATION SEAM

Mock ECS services

`EcsIdentity`, `EcsCharging`, `EcsWorkflow`, `EcsRoles`, `EcsReference`, `EcsFx`, `EcsTravelClassRegister` — the interface the vendor wires to live ECS.

Stack note. The reference prototype is built on Next.js (server actions), Prisma and SQLite. This spec describes **behaviour, data and logic** — not the prototype stack. On OutSystems, screens map to Screens/Blocks, server actions to Server Actions, the data model to Entities, and the mock services to integration (REST / service actions). Field types and rules below are platform-agnostic.

End-to-end flow

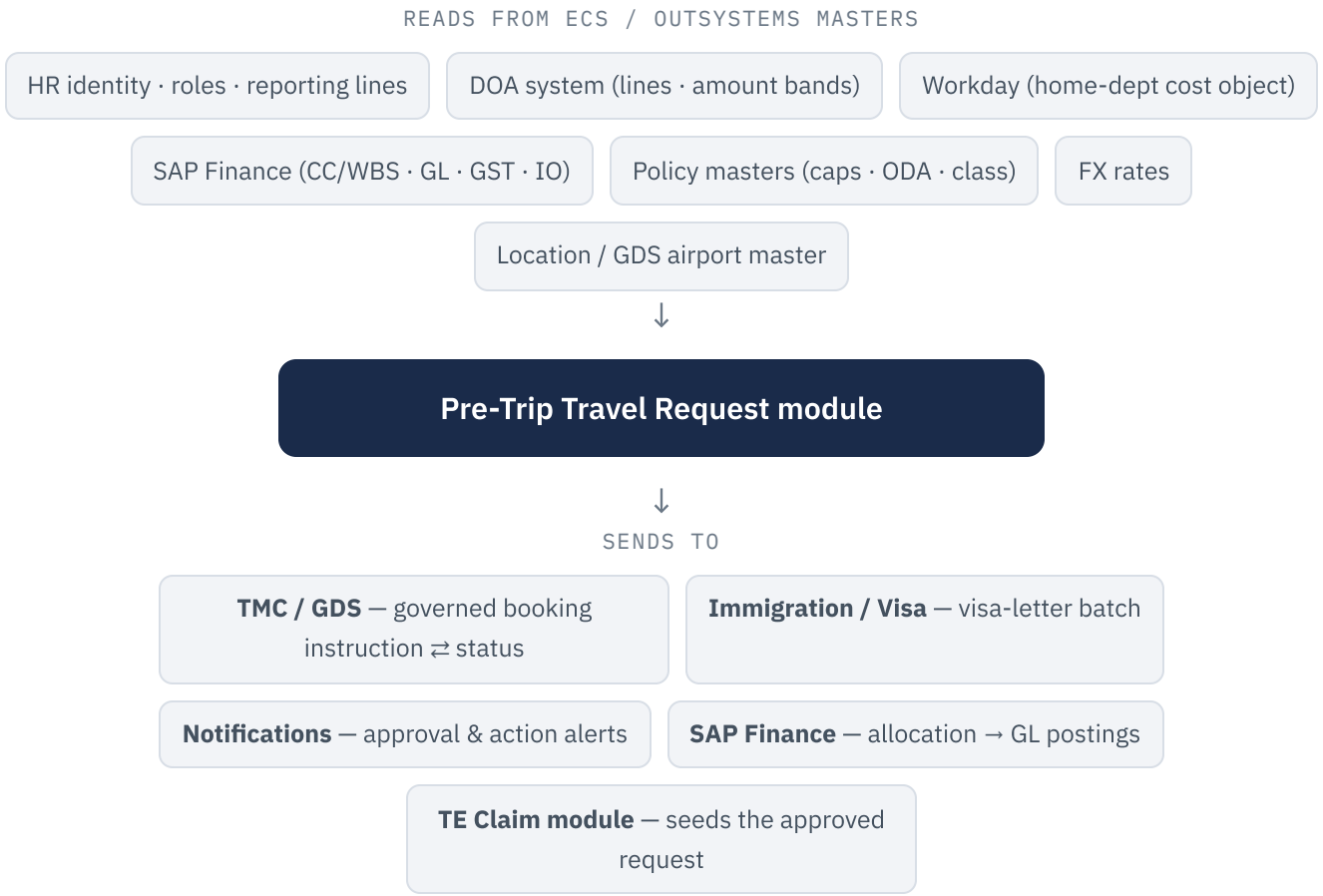
```
# happy path
Create —▶ Trip (purpose, booking, dates, itinerary)
       —▶ Estimates (airfare / accommodation / ODA / other — per traveller)
       —▶ Charging (cost object + allocation)
       —▶ Policy review (caps, class, exceptions)
       —▶ Review & submit (pick DOA, declare)
       —▶ Approval route (RO? ▸ Funding/Cross-BA? ▸ Exception? ▸ DOA)
       —▶ Approved + Travel Authorisation (TA-YYYY-NNNNNN)
       —▶ TMC hand-off (booking instruction) —▶ Booked
       —▶ Trip taken —▶ TE claim (pre-populated; DOA auto-granted within tolerance)
```

High-level architecture

Five layers. The module owns the presentation, domain logic and transactional store; it **reads** reference masters and talks to external systems through a thin integration seam.

LAYER	RESPONSIBILITY
Presentation	Wizard screens (Trip → Estimates → Charging → Policy → Submit), approver/DOA queue, TE claim, configuration. Server-rendered with client interactivity for the itinerary, charging splitter and live validation.
Domain logic	Approval-route engine, class-entitlement derivation, cost/ODA/accommodation computation, policy evaluation, charging roll-up, claim workflow & auto-grant.
Integration seam	ECS service interfaces (`EcsIdentity · EcsCharging · EcsWorkflow · EcsRoles · EcsReference · EcsFx · EcsTravelClassRegister`) + the TMC adapter and notification/visa hooks. The one place wired to live systems.
Transactional store	Requests, travellers, legs, estimates, allocations, policy checks, approval steps, bookings, claims, versions, audit, attachments.
Reference masters	Read-only ECS masters (identity, DOA lines, charging accounts, policy rates, class register, locations, FX, TMC providers) — never written by this module.

Interaction with other systems



SYSTEM	DIRECTION	WHAT FLOWS
ECS / ECS-Travel core	Read	Employee identity, appointments, roles, reporting lines; the request is created in the ECS context.
DOA system	Read	Per-department DOA lines (+ research line) and amount bands → the eligible DOA candidates and the tier.
Workday (HR)	Read	Each employee's home-department cost object (`defaultChargingCode`) — drives auto-allocate-by-department.
SAP / Finance	Read + Write*	Charging accounts (CC/WBS), GL accounts, GST codes, Internal Orders (read). The approved allocation drives the GL postings (*via the downstream claim / finance interface).
Policy masters	Read	Hotel caps, ODA daily rates, travel-class register (policy group, min hours, entitled class).
FX service	Read	Entry-date conversion to SGD, re-locked at submission.
Location / GDS airport master	Read (sync)	IATA airports — the same reference the TMC books against; synced from the TMC/GDS feed in production.
TMC	Outbound + Inbound	Governed booking instruction (field-filtered by the Integration Contract; one per traveller for a group) → booking status / traveller-action responses.
Immigration / Visa office	Outbound	Daily visa-letter batch for approved trips flagged as needing a visa letter (§4.13).
Notifications	Outbound	Approval requests, send-backs, booking actions, deadlines.
TE Claim module	Outbound (seed)	The approved request pre-populates the claim; the claim reconciles to the TA and auto-grants the DOA within tolerance.

2 Core design decisions

These decisions are binding — they shaped the data model, screens and logic below. Deviating from one changes multiple downstream behaviours.

DD-01 · Approval routing is configuration-driven

The route is computed, never hard-coded per user.

An evaluator takes the request context (amount basis, charging department, research flag, exceptions, group flag) and emits an ordered list of approval steps. Decision points are toggled by **Module Settings** (the existing ECS Configuration mechanism), not by code.

Why: the same engine serves the Travel Request and the TE claim; policy changes are config, not a release.

DD-02 · DOA is always required and traveller-selected

Every request ends at a DOA; the requestor picks the approver.

The DOA step is never dropped. The system offers the **eligible DOA candidates** (the charging department's DOA line from the amount-band tier upward, excluding travellers) and the requestor chooses — it is **not pre-selected**. Research routes use the research DOA line.

Why: the DOA holds financial authority; choosing (rather than auto-assigning) records intent and supports delegation.

DD-03 · RO is a configurable decision point

Reporting Officer step = ALWAYS · INDIVIDUAL_ONLY · NEVER.

Default `INDIVIDUAL_ONLY`: present for an individual request, dropped for a group (the group coordinator raises it). Resolved as the traveller's Reporting Officer; skipped if the RO is a traveller on the request (self-approval guard).

DD-04 · Travel class = derived entitlement, stored; an individual may override with justification

Entitlement is derived & stored; no picker on the trip form; an individual upgrade needs justification and raises an exception.

Class is derived from the itinerary — each flown leg's duration gives its entitled class and the **highest wins** (the **longest-duration leg** drives it, critical for multi-city). The derived entitlement is **persisted** (`entitledClassId` + `classBasis` on the request, per traveller for a group, and per leg). There is **no class picker on the trip form**; an **individual** request may select a **higher** class on the Add-Airfare estimate (`travelClassId`). A chosen class above entitlement: (1) **requires a justification** — a missing one is a **hard stop** that blocks submission (`CLASS_JUSTIFY`); and (2) raises a **"Travel class above entitlement" exception** (`CLASS`) that routes to the **Exception Approver when** `exceptionApproverRequired` **is on**, else is surfaced to the DOA. A **group** books each traveller at their own entitlement (no coordinator upgrade — DD-05).

Why: entitlement is deterministic from the register + itinerary, but a genuine upgrade is allowed with an auditable justification and the right approver — never silently.

DD-05 · Group travel is per-traveller for class & cost; shared for itinerary

One shared itinerary & dates; each traveller at their own entitlement.

Group members share the itinerary and dates (no per-traveller deviation). But **travel class is per traveller** (each at their own entitled class — the coordinator cannot justify an upgrade on another's behalf), and **all costs are per traveller**: airfare, ODA **and accommodation**. An upgrade for one person is handled by removing them from the group and raising a single request.

DD-06 · Accommodation is never shared — each traveller books, pays and claims their own room

No shared/apportioned hotel line.

Accommodation creates **one line per traveller** (own room, own nights), flowing to that traveller's own TE claim. "Shared — apportion" survives only for genuinely shared incidentals (e.g. a shared taxi under *Other*), where it is displayed apportioned across the group.

DD-07 · Charging allocation is per-expense-line and fully granular

Each cost line splits across one or more cost centres; lines are already per traveller.

ITEM-mode allocation gives every line a split of `{account, %}` rows (must total 100%). The request-level allocation that drives routing is the **roll-up** of net × % over every line × split. An **"Auto-allocate by traveller's Workday department"** preset fills each line from the attributed traveller's home-department cost centre.

DD-08 · TE claim DOA is auto-granted within a configurable tolerance

No re-approval when the claim matches the approved estimate.

The claim route mirrors the request (Verifier → Exception? → RO? → DOA). The DOA step is **auto-granted** from the approved Travel Request when the claim is within the configured tolerance (% and absolute SGD), adds no new expense type, introduces no new policy exception, and keeps the same charging department. No new expense type can be added at the claim.

DD-09 · One always-on itinerary; trip type is a preset, not a mode

Round-trip / One-way / Multi-city reshape the same leg builder.

The leg table is always shown. Trip type presets it: Round-trip = Outbound + mirrored Return, One-way = one leg, Multi-city = user-determined (add / reorder / remove). A round-trip may carry a personal segment. Main destination (for ODA defaults) = longest-stay leg. The **preset legs track the official dates** — Outbound ← official start, Return ← official end — so a later change to an official date moves a mirrored leg, while a leg the traveller set to a different date is preserved (applied on the form and re-asserted on save).

DD-10 · Booking arrangement and TMC booking method are separate

What arranges the trip vs how the TMC fulfils it.

Booking arrangement: Auto (route by policy to a TMC) · a specific TMC provider · Self-booked · Host-arranged · No booking. **TMC booking method** (TMC channel: online / agent-assisted / offline) applies only when arranged via a TMC. The resolved TMC is shown read-only on the trip form.

DD-11 · The airport master is the TMC's booking source

One IATA location master, shared.

From/To on every leg read the shared ECS Location master (IATA airports). Each leg's code is carried verbatim into the TMC booking instruction. In production this master is synced from the TMC/GDS airport feed.

DD-12 · A Travel Request has no "Reject"

Dissent = Send Back.

An approver who disagrees **sends the request back** (→ Sent Back → Draft) to be revised and resubmitted. There is no terminal reject on the request. (The TE claim keeps standard claim controls.)

DD-13 · A group may mix employees and guests; its size is capped at the TMC limit

Guest (non-employee) members fly Economy, are host-settled, and never claim; a configurable limit caps the booking.

A group booking can include **guest members** (external speakers / collaborators with no ECS/HR profile) alongside employees. A guest has no entitlement profile so defaults to **Economy**, is **charged to the host (requestor) department**, and is **settled centrally by TMC direct-billing** — guests have no login and **file no TE claim**. A guest carries **no share of any shared incidental** and receives **no ODA** (neither is host direct-billable — both are blocked and validated). Mixing is **configurable** (`groupGuestMixAllowed` , default on); when off, a group is employees-only and guests use a separate request. The total membership may not exceed `groupMaxTravellers` (default **10**). Full logic in §14.

Why: real group trips routinely carry an invited guest whose cost the host absorbs; but a guest has only one settlement path (direct-bill), so any cost it can't carry there — shared incidentals, ODA — must never be attributed to it.

DD-14 · One live authorisation per trip — a duplicate is blocked, not silently doubled

A traveller can't be on two trips at once; a re-raise must cancel/amend the original first.

A request may not be submitted when a **live** request (Submitted · Pending* · Approved · Amendment In Progress) already **shares a traveller** and **overlaps the official dates**. Same destination = a **true duplicate** → **hard block** (submit disabled; the traveller cancels or amends the original first); a different destination with overlapping dates = a **date clash** → **warning** (possible double-booking). Groups match per shared employee; guests are excluded (a guest may legitimately appear on more than one host's trip).

Why: after a failed booking a traveller tends to re-raise; without this guard two live Travel Authorisations can exist for one trip (double-booking, double-spend, orphaned TA). The guard makes "cancel/amend before re-raise" enforceable.

DD-15 · A failed booking recovers by amend-and-reroute or cancel-and-reraise; the OBT reports status, it does not reject

The OBT/TMC is a fulfilment channel, not an approver — "failed" is inferred from not-issued within the booking window.

The OBT (e.g. Cytric, Trip.Biz) returns booking **status** (confirmed / issued / changed / cancelled / refunded), not an approval verdict. A booking that does not issue within the window is **Booking Failed**. Recovery offers, gated by the booking window (deadline / TA validity): **re-send** (retry as-is), **amend & re-route** (reopen to Amendment In Progress, re-derive the route — the single-TA path), or **cancel & re-raise** (the original goes terminal, freeing the overlap for a fresh request under DD-14). A material booking deviation (fare over tolerance) likewise routes to re-approval — it is never the OBT "sending the request back".

Why: the fare/seat may not survive an approval round-trip, so recovery must consider the window; and authority stays with the human approver, never the booking tool.

DD-16 · Estimate inputs default to the policy figure, and are scoped to entitlement

Accommodation rate defaults to the city's cap (editable); ODA auto-computes; a guest estimate is airfare + accommodation only.

An estimate pre-fills from the governing policy figure so the common case needs no lookup: the **accommodation quoted rate defaults to the selected city's hotel cap** and **follows a city change**, but stays **editable** for an actual quote; **ODA** auto-computes from the policy rate × eligible days over the official range. Entitlement scopes what can be added: for a **guest** the **ODA form is disabled** (not a guest entitlement — §8), so a guest estimate is **airfare + accommodation only**; in a group the ODA traveller picker offers **employees only**.

Why: defaulting to the cap removes a manual lookup and anchors estimates to policy, while keeping the quote editable; scoping stops a guest accruing a cost with no settlement path.

3 Data model

Transactional entities (new). Status/type fields are constrained strings; allowed values are in §4 and inline. Reference masters are in §13.

TravelRequest — header & lifecycle

FIELD	TYPE	NOTES
requestNumber	string, unique	TR-YYYY-NNNNNN
authorisationNo	string, unique?	TA-YYYY-NNNNNN — set on final approval
status / bookingStatus	enum string	Approval + booking state machines (§4)
isGroup	bool	Multi-traveller request
requestorId / travellerId	employee id	Creator may ≠ traveller; `GUEST` for non-employee
travellerType, guest*	string	EMPLOYEE | GUEST (+ guest name/email/org)
additionalApproverId	employee id?	Optional traveller-selected extra approver
selectedDoaId	employee id?	Traveller-chosen DOA (not pre-selected)
purposeId, description	ref / text	Travel purpose (drives research flag)
tripType	enum	ROUND_TRIP | ONE_WAY | MULTI_CITY
destCountry, destCity	ref	Derived from longest-stay leg on save
startDate, endDate	date	Official travel window
personalStart, personalEnd	date?	Personal extension (excluded from ODA/accommodation)
eventStartDate, eventEndDate, invitationRef	date / text	Event is a segment within the official window
bookingArrangement	enum	AUTO | TMC-<id> | SELF_BOOKED | HOST_ARRANGED | NO_BOOKING
bookingMethod	enum?	TMC channel — only when arranged via a TMC
tmcProviderId	ref?	Resolved/selected TMC (null = auto by policy)
travelClassId, entitledClassId, classBasis	ref / text	Request-level class = derived entitlement
chargingMode	enum	MAIN | CLAIM | ITEM (allocation entry mode)
approvalAmountSgd, bookingDeadline, authorisationExpiry	num / date	Derived approval snapshot (§13.3/§13.4)

FIELD	TYPE	NOTES
visaLetterRequired, highRiskAck, personalAck	bool	Declarations / flags

TravelRequestTraveller

FIELD	NOTES
employeeId	Group member
isRequestor	Coordinator flag
sharePct	Share of NTU-funded cost
entitledClassId / chosenClassId	Per-traveller class (DD-05)
classBasis / classJustification	Derivation / upgrade reason
confirmed, confirmedAt	Inclusion confirmation

ItineraryLeg

FIELD	NOTES
travellerId	null = shared group leg
seq	Order
originCode / destCode	IATA (shared master)
departDate / arriveDate	Within travel window
durationHours	Derived from dest city → drives class
nights	Stay at destination
isPersonal	Personal segment (excl. ODA)
entitledClassId / chosenClassId	Per-leg entitlement

EstimatedExpense — one cost line (per traveller)

FIELD	TYPE	NOTES
travellerId	employee id?	Attributed traveller; null = requestor/shared owner
category / expenseTypeId	enum / ref	AIRFARE | ACCOMMODATION | ODA | CONFERENCE | OTHER
currency, foreignAmount, sgdAmount	num	Converted at entry-date FX, re-locked at submit
sponsorForeign, sponsorSgd	num	Expected sponsorship (reduces NTU-funded)
estimateBasis	enum	QUOTED | CAP | LOWER | RATE
isShared, shareMap	bool / JSON	Only for genuinely shared incidentals (DD-06)
chargingCode	ref?	Single per-line account (ITEM mode)
allocJson	JSON?	Per-line **multi-account split** `[{code,pct,io}]` (DD-07)
originCode, destCode, proposedClassId, fareCeiling	—	Airfare-specific
+ AccommodationEstimate	1:1	city, nights, personalNights, quotedNightly, capNightly, budgetedNightly, exceptionOutcome
+ ODAEstimate	1:1	country, city, arrive, depart, eligibleDays, personalDays, dailyRate

ChargingAllocation — request-level roll-up

FIELD	NOTES
chargingCode, chargingType	CC / WBS id
companyCode, businessArea	SAP org
percent, amountSgd	Share of allocated amount
internalOrder, isMain, isResearch	SAP IO; main account flag; research

Drives DOA routing (highest-share department), cross-BA concurrence and the finance breakdown.

ApprovalStep

FIELD	NOTES
seq, roleType	RO | Additional | FundingOwner | Exception | DOA | ResearchDOA
approverId	Resolved approver
status, decidedAt, comments	Pending | Approved | SentBack
autoGranted	TE claim DOA auto-grant (DD-08)

TravelExpenseClaim & TEExpenseLine

ENTITY · FIELD	NOTES
Claim.claimNumber, claimantId, status	Draft → Submitted → … ; one claimant
Claim.lessCorpCard, lessPersonal	Deductions from reimbursable total
Claim.chargingMode, chargingMainCode, chargingJson	Allocation snapshot (same editor as the request)
Claim.approvalSteps[]	Verifier → Exception? → RO? → DOA (auto-grant)
Line.expenseTypeId, treatment	LOCKED | EDITABLE | REFERENCE | EXCLUDED (from the TR estimate)
Line.approvedSgd, bookedSgd, actualSgd, varianceSgd	Pre-populated from the TR; variance tracked
Line.receiptType, receiptOk	LOCAL_GST | FOREIGN_NO_GST | NO_RECEIPT
Line.chargingCode, allocJson	Per-line (multi-)account split, as the request

4 Status model & lifecycle

Two independent state machines on an approved request: **approval status** and **booking status**. Invalid transitions must not be offered.

Approval status transitions

FROM	ALLOWED TO
`Draft`	Submitted · Withdrawn
`Submitted`	Pending Additional · Pending Exception · Pending DOA
`Pending RO Approval`	Pending DOA · Pending Exception · Sent Back
`Pending Additional Approval`	Pending Exception · Pending DOA · Sent Back
`Pending Exception Approval`	Pending DOA · Sent Back
`Pending DOA Approval`	`Approved` · Sent Back
`Approved`	Amendment In Progress · Cancelled · Expired · Closed
`Amendment In Progress`	Approved · Sent Back · Cancelled
`Sent Back`	Draft · Withdrawn
`Expired`	Pending RO · Pending DOA · Cancelled (re-validate)

No Reject (DD-12). There is no transition to a terminal "Rejected" on a Travel Request — disagreement routes through **Sent Back** → **Draft**.

Booking status (post-approval, independent)

`Not Sent` `Sent to TMC` `Received by TMC` `Booking In Progress` `Traveller Action Required` `Partially Booked` `Booked` `Travel Completed` `Self-Booked` `Booking Failed` `Cancelled`

5 Screens

Each step is a stage in the request wizard (Traveller → Trip → Estimates → Charging → Policy Review → Submit), plus the approver, claim and configuration surfaces. Fields, layout and behaviour below; validation is consolidated in §6.

5.1 Travel Request Dashboard

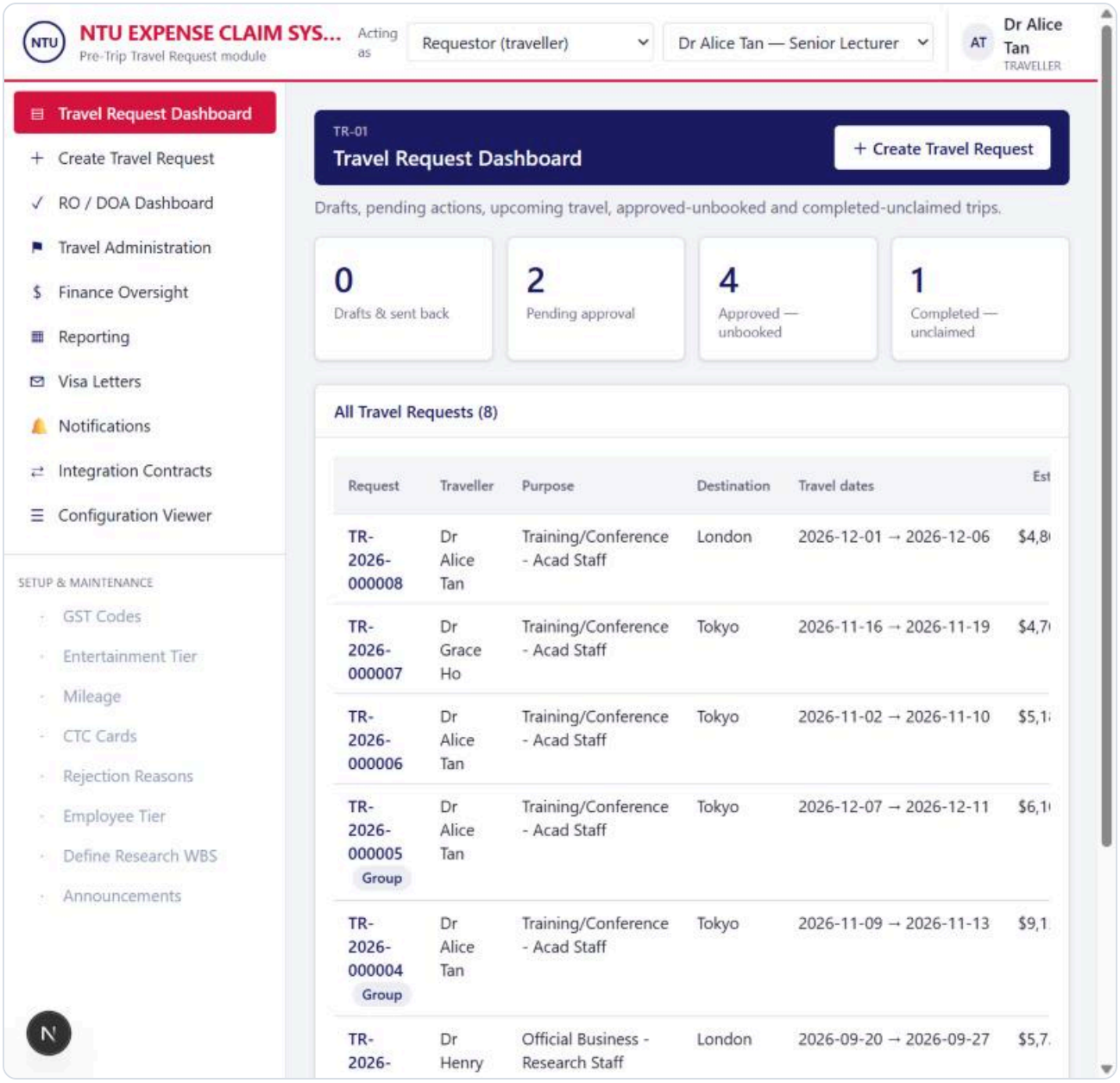
/dashboard

Entry point. Buckets of the user's requests with a load-demo seed for non-production environments.

Content

- Summary tiles: **Drafts & sent back**, **Pending approval**, **Approved — unbooked**, **Completed — unclaimed**.
- All-requests table: Request · Traveller · Purpose · Destination · Travel dates · Est. NTU cost · Exceptions · Status · Booking.
- Group requests tagged `Group` ; row links to the detail page.

PROTOTYPE UI



Dashboard — summary tiles (drafts, pending, approved-unbooked, completed-unclaimed) and the all-requests table.

5.2 Request detail / Traveller

`/requests/{id}`

Request header, approval status, workflow visual, and (for groups) the traveller roster.
Read-only summary of the approved request; the editing wizard steps follow.

Header summary

Purpose · Destination · Dates	Trip header
Travel class	Individual: entitled class. **Group:** "Each traveller flies at their own class entitlement" + a **Class column in the roster**
Booking arrangement · TMC	Resolved TMC provider shown
Primary charging account	Main CC/WBS + %

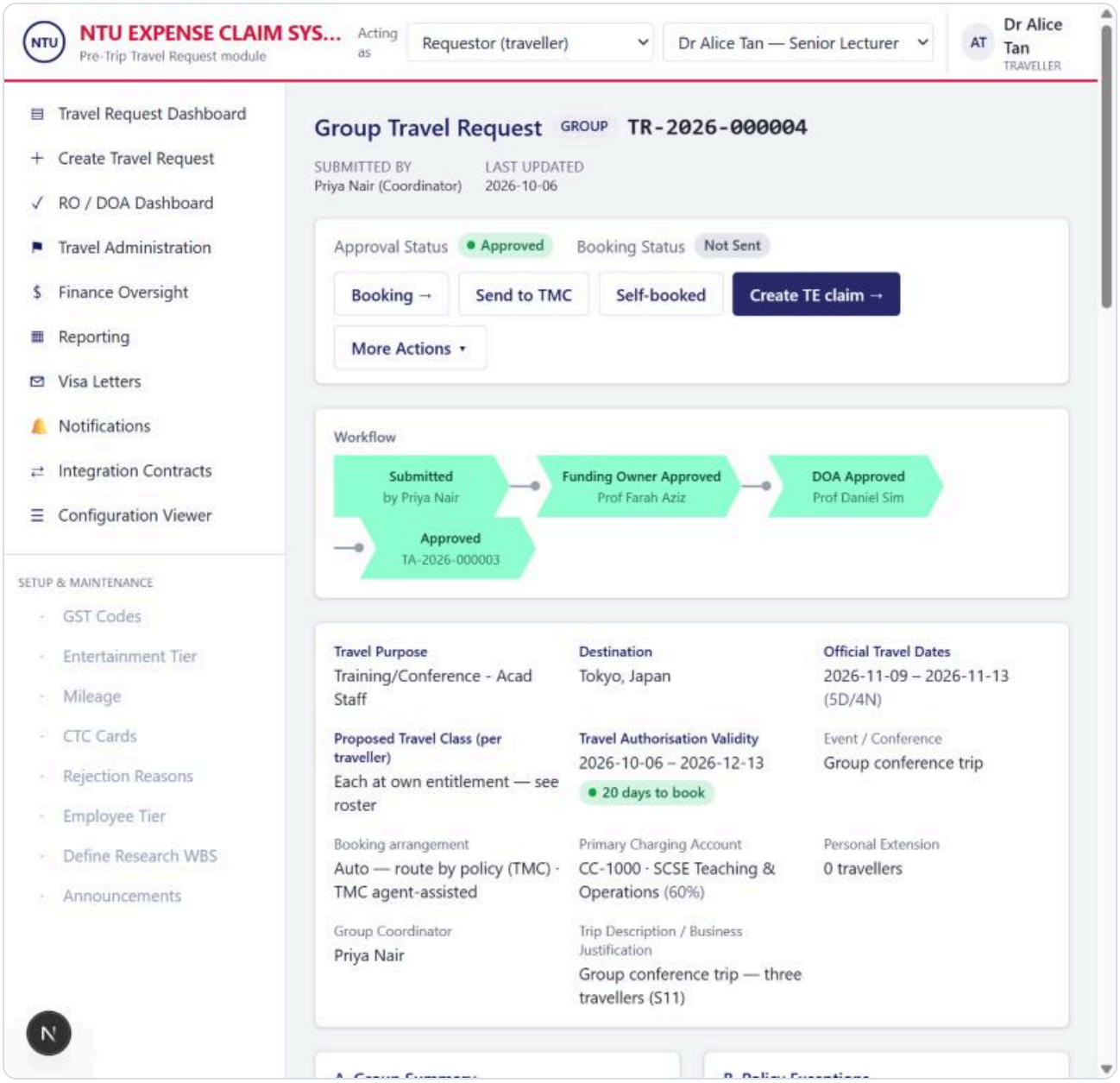
Group roster (isGroup)

· Traveller · Department · **Class** · Travel dates · Est. cost · Exception · Booking — one row per traveller, each at their own entitled class.

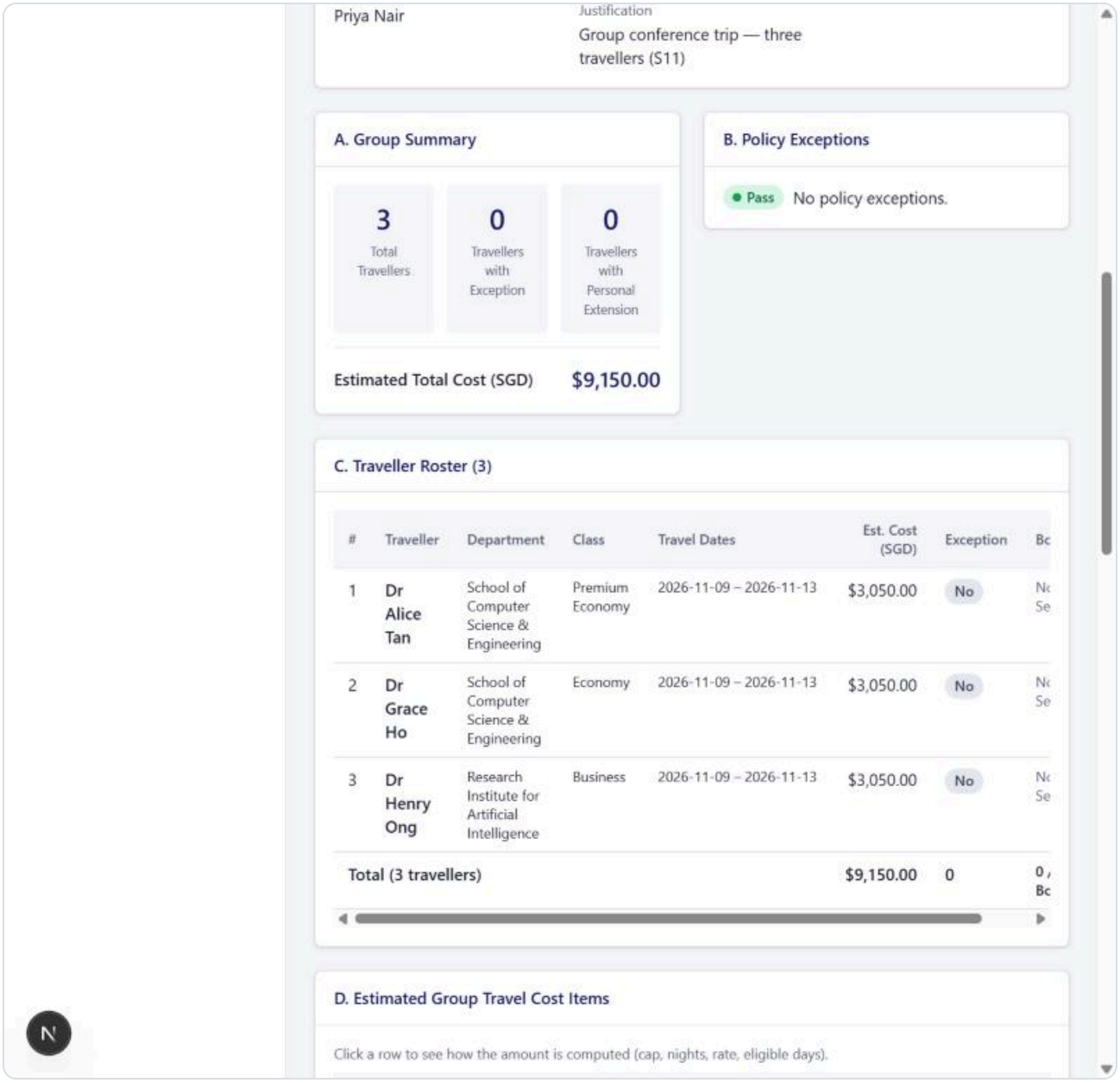
Actions (post-approval)

Booking · Send to TMC · Self-booked · Create TE claim · More actions (amend, cancel).
Workflow chevrons show the realised route with approver names.

PROTOTYPE UI



Group request detail — approval status, realised workflow (Submitted → Funding Owner → DOA → Approved + TA number), and the per-traveller travel-class statement.



Group summary + Traveller Roster — each member with their department, **own entitled class** (Premium Economy / Economy / Business) and cost share.

5.3 Trip details

`/requests/{id}/trip`

Purpose, booking arrangement, dates and the always-on itinerary. **No class picker here** — the class is derived and shown read-only; an individual's optional higher-class selection (with justification) is made on the Add-Airfare estimate (§5.4), not on this form (DD-04).

Trip Purpose & Justification

`Travel purpose`	Reference list; research purposes flag the research route
`Booking arrangement`	Auto (TMC) · provider · Self-booked · Host-arranged · No booking (DD-10)
`TMC booking method`	TMC channel — enabled only for TMC arrangements
`TMC (resolved)`	**Read-only** — resolved provider from the arrangement (e.g. "CTC Travel (Asia-Pacific)")
`Description, Invitation ref, Visa letter`	Justification + §4.13 visa flag

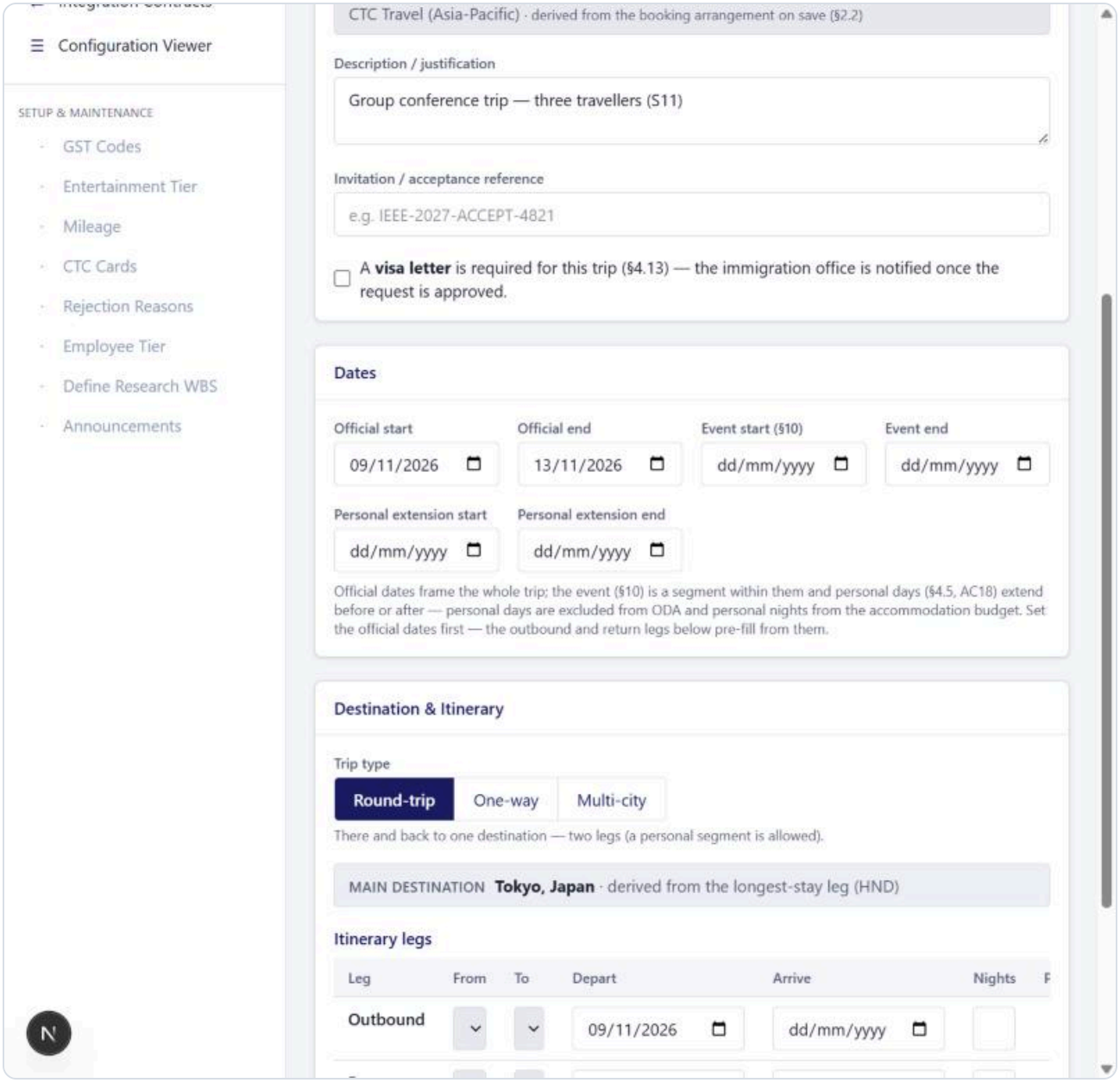
Dates — ordered Official → Event → Personal

Official start/end frame the whole trip; the **event** (§10) is a segment within them; **personal** extension (§4.5) extends before or after. The outbound & return legs pre-fill from the official dates.

Destination & Itinerary (DD-09)

- **Trip type** selector presets the leg table (Round-trip 2 · One-way 1 · Multi-city N).
- **Main destination** indicator (read-only) — derived live from the longest-stay leg.
- **Itinerary legs**: Leg · From · To · Depart · Arrive · Nights · Personal (+ add/reorder/remove for multi-city). From/To = IATA master, shown "CODE — Airport name".

PROTOTYPE UI



Trip form — **resolved TMC** (read-only), Dates ordered Official → Event → Personal, the trip-type preset, the derived **Main destination** and the always-on itinerary legs. No class picker here — class is derived and read-only; an individual upgrade is made on the Add-Airfare estimate (DD-04).

5.4 Estimated travel costs

`/requests/{id}/estimates`

Per-traveller cost lines, pre-filled from the trip. Four add-forms: Airfare, Accommodation, ODA, Other/Conference.

List (group = collapsible, one row per expense type)

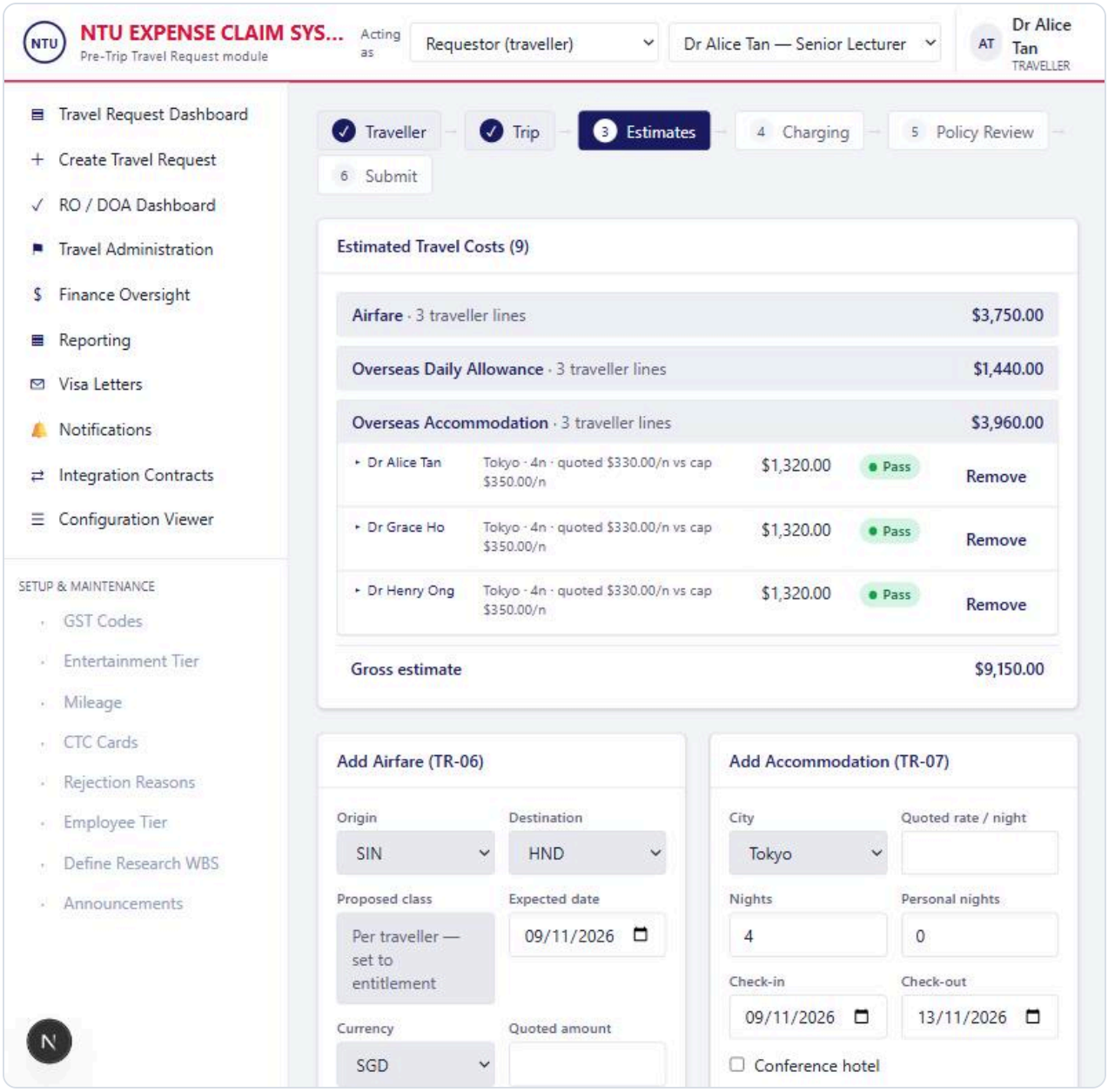
Grouped by cost item with a subtotal; expand to the per-traveller rows. Individual = flat table. Gross estimate → less sponsorship → NTU-funded.

Add-forms — group behaviour

Traveller selector	Individual traveller **or "All travellers"** — one line per traveller in a single submit
Airfare · Proposed class	Group: **read-only** "per traveller — set to entitlement"; line stamped with each traveller's class
Accommodation	**Per traveller** (own room) — no Shared checkbox (DD-06)
Other / Conference	May be **Shared — apportioned** for genuinely shared incidentals

The accommodation **rate defaults to the selected city's hotel cap** (follows a city change, editable) and shows quoted/night vs cap, flagging an exception over the cap. **ODA** is auto-computed from policy rate × eligible days over the official range (personal days excluded). For a **guest** the **ODA form is disabled** — a guest estimate is airfare + accommodation only (§8, DD-16).

PROTOTYPE UI



Group estimates — collapsible, one row per cost item; Overseas Accommodation expanded to **one room per traveller** (DD-06). Add-forms show the per-traveller class and the Traveller / All-travellers selector.

5.5 Charging & allocation

`/requests/{id}/charging`

ECS 4-level cascade + allocation. Drives DOA routing.

Estimated Cost Lines

Type · **Traveller** (group) · Detail · Gross · Sponsor · NTU-funded, with the NTU-funded total.

Main charging account — cascade

Company Code → Business Area → CC/WBS → Charging Account. Defaults to the traveller's profile cost object. Closed accounts cannot be charged.

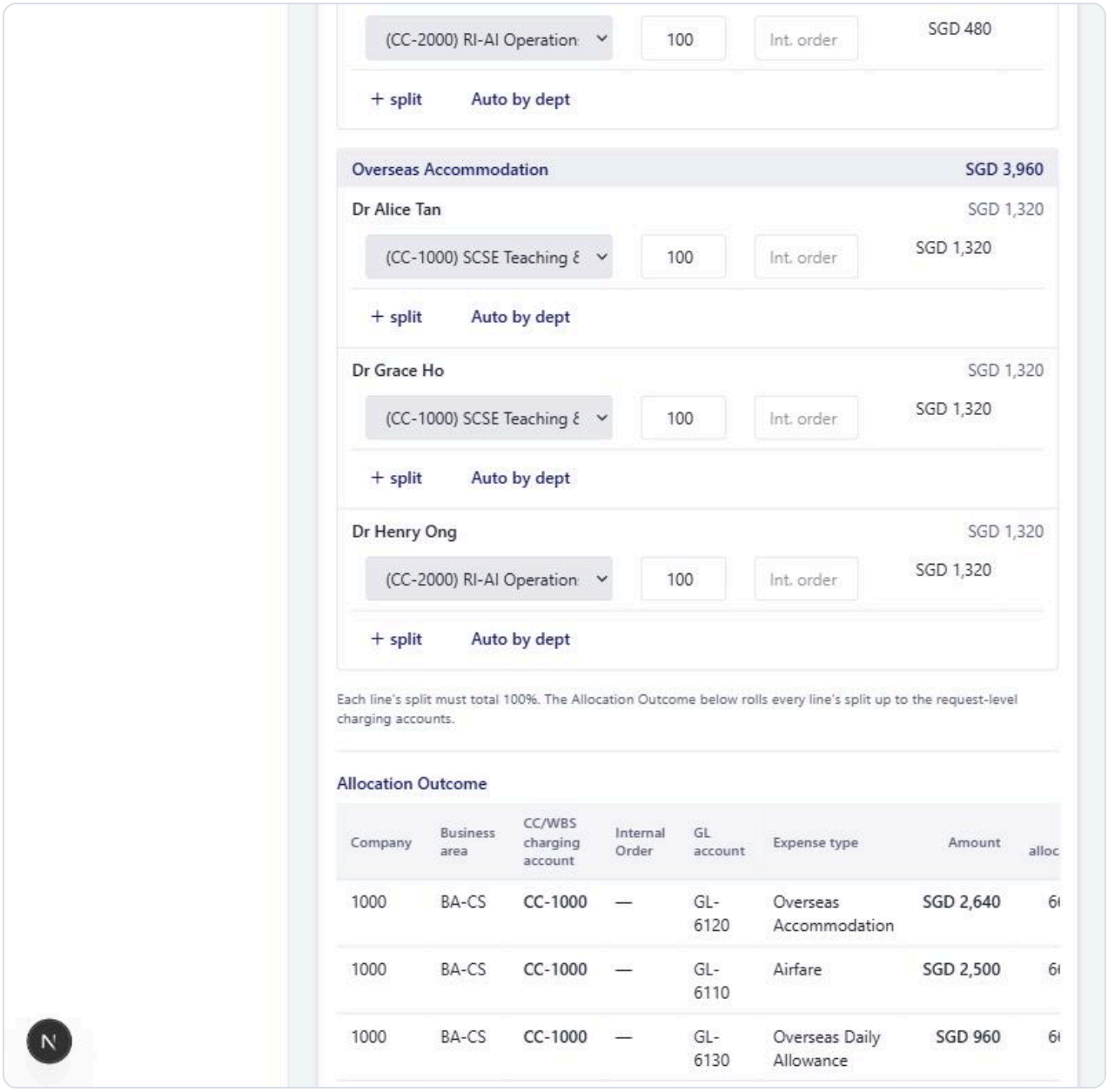
Cost allocation modes

- **MAIN** — whole request to the main account.
- **CLAIM** (Allocation at Claim/Item Level) — a %/amount split of the whole request across accounts.
- **ITEM** (Allocation per Expense Item) — **per-line splitter** grouped by cost item; each line splits across 1+ accounts (% to 100). **Auto by dept** per line + **Auto-allocate all by traveller's department** (DD-07).

Allocation Outcome (TR) / Accounting Entries (TE)

Computed GL postings per account × expense type: Company · BA · CC/WBS · Internal Order · GL · Expense type · Amount · % · Main acct · GST. Must total 100% before submit.

PROTOTYPE UI



ITEM-mode allocation auto-allocated by department: each per-traveller room routes 100% to that traveller's dept (Alice/Grace → CC-1000, Henry → CC-2000), and the **Allocation Outcome** rolls up the GL postings.

5.6 Policy review

`/requests/{id}/policy`

Runs the common + pre-trip rules and lists outcomes: `Pass` `Exception` `Hard stop`. Hard stops block submission; exceptions are carried to the Exception Approver or surfaced to the DOA per config.

5.7 Review & submit

/requests/{id}/review

Approval Route (visual) — above the Trip card

Chevron preview of the computed route. The requestor **selects the DOA** (required; from eligible candidates; not pre-selected) and may add one Additional Approver — the chevrons update live.

Trip card

Purpose · Destination · Dates · Travel class (group = per-traveller statement) · Booking arrangement · **TMC** · Traveller. Multi-city shows the full leg list.

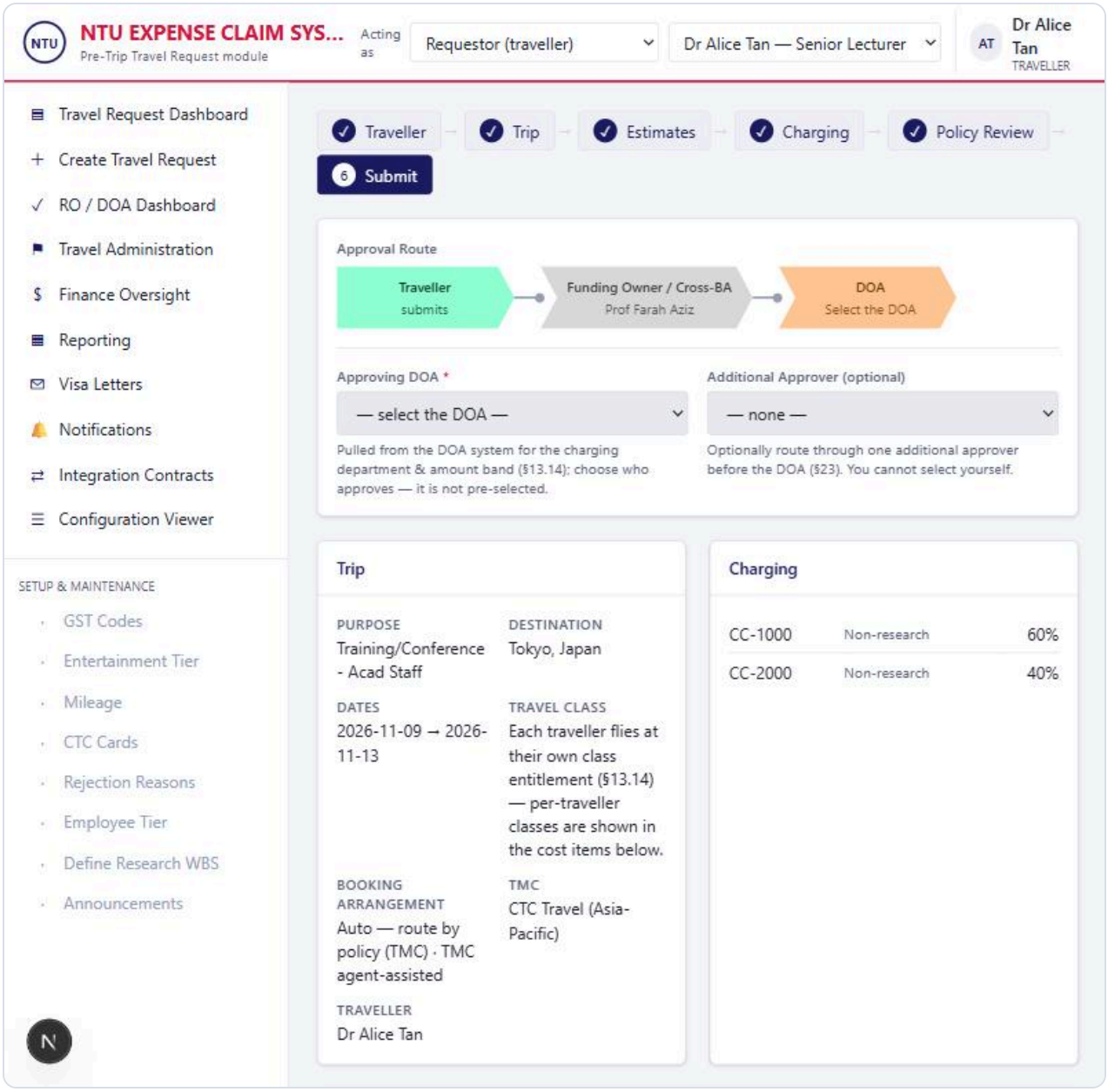
Estimated cost items

Grouped by type (group = expandable to per traveller): Gross → less sponsorship → Estimated NTU-funded → Approval amount (per basis) → approved booking ceiling.

Declaration & submit

Official-purpose + conflict-of-interest declaration; high-risk acknowledgement when applicable. Submit disabled while any hard stop remains or no DOA is chosen.

PROTOTYPE UI



Review & submit — the computed **Approval Route** chevrons (Traveller → Funding Owner / Cross-BA → DOA), the required **DOA selector** (not pre-selected), and the Trip card with the resolved TMC and per-traveller class statement.

5.8 Approver / DOA dashboard

`/approvals`

Per-role queue (acting-as role selector). Each pending step shows the request, the approval amount, exceptions, and the right-aligned numeric data. Decisions: **Approve** or **Send Back** (with reason). A high-risk request requires an approver acknowledgement.

5.9 TE claim (Create from TR)

`/claims/{id}`

Pre-populated from the approved request so the claimant does not re-key. Header (report type, travel dates, deductions), expense lines (pre-filled, treatment-controlled), and the same charging editor.

Pre-population

Each estimate line seeds a claim line (approved amount, charging), with per-line treatment LOCKED / EDITABLE / REFERENCE / EXCLUDED. **No new expense type** may be added at the claim.

Workflow

Verifier → Exception? → RO? → **DOA (auto-granted within tolerance)** — see §10.

5.10 Configuration / Module Settings

`/settings · /config-viewer`

The existing ECS Configuration mechanism — a single settings row drives routing switches, thresholds, caps basis, declarations and tolerances. Full catalogue in §11.

6 Validation rules

Every rule is enforced **server-side** (authoritative); the client mirrors them for instant feedback and disables the action while a blocking rule fails.

Trip & dates

RULE	ENFORCEMENT
Travel purpose, ≥1 itinerary leg with a destination, travel dates, booking arrangement required	Block save
Official end ≥ official start	Block
Each leg: arrive ≥ depart	Block
Legs chronological — leg `n+1` depart ≥ leg `n` depart	Block
Each leg departs within the travel window (widened by a personal extension on either side)	Block
Personal extension: end ≥ start, and the window **does not overlap** the official dates (before start or after end only)	Block
Event dates within the official window (§10) — event is a segment of the trip	Block

Charging

- Each cost line's split must total **100%** (ITEM mode); the request-level allocation must total 100% before submit.
- A closed/inactive account cannot be charged.
- Auto-by-department falls back to the main account and flags any traveller with no mapped department cost centre.

Submission

- All **hard stops** resolved (AC06).
- A **DOA is selected** when candidates exist (DD-02).
- High-risk destination acknowledgement checked when applicable (§4.8).
- **No duplicate live request (DD-14):** submission is blocked when another live request (Submitted · Pending* · Approved · Amendment In Progress) shares a traveller, overlaps the official dates and has the **same destination**. A date overlap with a **different** destination is surfaced as a warning (possible double-booking), not a block.
- **No guest-attributed shared/ODA cost (§8):** a shared line whose shareMap names a guest, or an ODA line attributed to a guest, is a hard block.
- Status transition is in the allowed set (§4); self-approval prevented (an approver who is a traveller is skipped/escalated).

TE claim

- No expense type absent from the approved request.
- A line claimed with `NO_RECEIPT` > 0 raises a new exception (blocks auto-grant).
- Claim over the approved amount beyond tolerance raises a new exception.

7 Program logic — approval routing

The route builder consumes the request + settings and emits ordered steps. Order: Additional → RO → Funding Owner → Cross-BA concurrence → Exception → DOA.

```
function buildRoute(req, settings):
  steps = []; travellers = req.isGroup ? req.travellers : [req.travellerId]
  dept = highestChargingDept(req)          # department bearing the largest %
  research = isResearch(req) and not settings.sameRouteResearch

  # 1 · Additional Approver (traveller-chosen, optional) — self-approval guard
  if req.additionalApproverId and notTraveller:
     steps += ADDITIONAL(req.additionalApproverId)

  # 2 · RO decision point (DD-03)
  roNeeded = settings.roRequirement == "ALWAYS"
          or (settings.roRequirement == "INDIVIDUAL_ONLY" and not req.isGroup)
  if roNeeded and RO(traveller) notTraveller: steps += RO

  # 3 · Funding Owner (cross-charge requiring owner concurrence)
  if anyAllocation.crossCharge and fundingOwner: steps += FUNDING_OWNER

  # 4 · Cross-business-area concurrence (§18)
  for each non-primary BA whose share > settings.crossBaThresholdSgd:
     steps += FUNDING_OWNER(thatBA)            # skip duplicates / travellers

  # 5 · Exception approver — only if a policy exception exists AND configured on
  if hasException and settings.exceptionApproverRequired: steps += EXCEPTION

  # 6 · DOA (always) — tier from the amount band, line per research route (DD-02)
  band = bandForAmount(approvalAmount)
  doaId = req.selectedDoaId   # traveller-chosen; else pickDoa(line, band.tier, escalate pas
  steps += (research ? RESEARCH_DOA : DOA)(doaId)
  return steps
```

DOA candidate derivation (what the requestor chooses from)

```
function doaCandidatesFor(req, approvalAmount):
  line = research ? dept.researchDoaLine : dept.doaLine
  band = bandForAmount(approvalAmount)
  tier = min(band.doaTierIndex, line.length-1)
  return line.slice(tier)                    # required tier upward (higher authority can al
         .filter(id not in travellers)          # never a traveller (self-approval, §13.5)
```

Status derivation. The request's `Pending *` status is set from the first pending step's role (RO → Pending RO Approval, Additional → Pending Additional, Exception → Pending Exception, else Pending DOA).

8 Program logic — travel class entitlement

Entitlement is derived from the **Travel Class Register** (per-employee policy group, entitled class, minimum flight-hours threshold, effective dates) across the whole itinerary.

```
function entitledForDuration(employee, hours, onDate):
  entry = register(employee, onDate)
  return entry and hours >= entry.minHours ? entry.entitledClassId : ECONOMY

function entitledForItinerary(employee, legs, onDate):   # DD-04
  perLeg = [ entitledForDuration(employee, leg.durationHours, onDate)
             for leg in legs if not leg.isPersonal ]
  return highest-rank(perLeg)                  # longest-duration leg drives the class
```

- **Individual:** the derived entitlement is **stored** (`entitledClassId` + `classBasis`); the chosen class (`travelClassId`) defaults to it but may be **overridden to a higher class** on the Add-Airfare estimate.
- **Group (DD-05):** compute per traveller and store on each `TravelRequestTraveller.chosenClassId / entitledClassId` . Each airfare line is stamped with that traveller's class; lines re-sync if the itinerary changes. No coordinator upgrade.
- Leg duration is derived server-side from the destination city's flight-hours — not trusted from the form.

Upgrade control (chosen > entitled)

```
if rank(chosen) > rank(entitled):                  # per traveller
  raise EXCEPTION "Travel class above entitlement"   # code CLASS
  if not justification:
     raise HARD_STOP "Class upgrade justification required"  # code CLASS_JUSTIFY — blocks s
# the CLASS exception then routes to the Exception Approver when exceptionApproverRequired,
# otherwise it is surfaced to the DOA (§7 routing, DD-01).
```

So an individual **may** fly above entitlement, but only with a stored **justification** (absent = hard stop) and the **right approver** — the Exception Approver or the DOA per `exceptionApproverRequired` . Nothing is silent; the derived entitlement is always retained for comparison.

9 Program logic — charging & allocation

Per-line split → request-level roll-up (ITEM mode)

```
function resolveItem(lines):
  charges = []
  for line in lines:
     allocs = line.allocJson or [{code: line.mainAccount, pct: 100}]
     line.save(allocJson = allocs.length>1 ? allocs : null)   # persist genuine splits
     for a in allocs: charges += {code: a.code, net: line.net * a.pct/100}
  rows = rollupByCode(charges)          # sum net per account → % of total
  persist ChargingAllocation(rows)        # drives routing — unchanged contract
```

Auto-allocate by traveller's Workday department (DD-07)

```
function deptAllocsFor(line):
  shares = line.isShared
         ? (line.shareMap or equalSplit(req.travellers))   # shared incidental
         : [{ eid: line.travellerId, pct: 100 }]             # per-traveller line
  byCode = {}
  for s in shares:
     cc = employee(s.eid).defaultChargingCode               # Workday home-dept cost object
     byCode[cc] += s.pct                                     # aggregate same-dept travelle
  return byCode  # → one {account, %} row per department, editable after fill
```

Because accommodation (and airfare, ODA) are now **per traveller** (DD-06), each line auto-allocates 100% to that traveller's department — **no split**. A shared incidental splits across departments by share. The highest-share department derives the DOA (§13.14).

10 Program logic — TE claim & DOA auto-grant

The claim route reuses the same configurable rules; the DOA step is auto-granted when the claim matches the approved request.

```
function doaAutoGrant(claim, req, settings):   # DD-08
  if req.status not in [Approved, Closed]:  return false  "not approved"
  if not req.authorisationNo:               return false  "no TA"
  over = claimTotal(claim) - approvedAmount(req)
  withinTol = over <= settings.teAutoGrantToleranceAbsSgd
          and over <= approvedAmount * settings.teAutoGrantTolerancePct/100
  if not withinTol:                         return false  "beyond tolerance"
  if claimHasNewExpenseType(claim, req):    return false  "new expense type"
  if claimHasNewException(claim, req):      return false  "new exception"
  if claimChargingDept != reqChargingDept:  return false  "DOA changed"
  return true  # DOA step marked Approved · autoGranted, cites the TA
```

New exception = over the approved amount beyond tolerance **or** any line claimed without a receipt. The Verifier (Travel Administrator) is always first; the Exception and RO steps follow the same config as the request.

11 Configuration reference — Module Settings

Single settings row (the ECS Configuration mechanism). Runtime-togglable; changes affect routing and derivation immediately.

SETTING	VALUES · DEFAULT	EFFECT
roRequirement	ALWAYS · **INDIVIDUAL_ONLY** · NEVER	RO decision point (DD-03)
groupTravelEnabled	bool · **true**	Enables one-request-many-travellers group requests (DD-05)
groupMaxTravellers	int · **10**	Group booking size limit incl. guests — TMC group-fare cap (DD-13)
groupGuestMixAllowed	bool · **true**	May a group mix employees and guests? Off = employees-only; guests use a separate request (§8, DD-13)
exceptionApproverRequired	bool · **true**	Insert an Exception step, else surface to DOA
sameRouteResearch	bool · **false**	Treat research like non-research (no Research DOA line)
approvalAmountBasis	GROSS · **NET_NTU** · MAJOR_COST · ALLOCATION	Which amount drives the band / DOA tier
crossBaThresholdSgd	int · **500**	Cross-BA concurrence threshold (§18)
teAutoGrantTolerancePct	int · **10**	Claim DOA auto-grant tolerance (% over)
teAutoGrantToleranceAbsSgd	int · **500**	Claim DOA auto-grant tolerance (abs SGD)
hotelEstimateBasis	QUOTED · CAP · **LOWER**	Accommodation budget basis
expenseScope	MAJOR_ONLY · **ALL**	Whether incidental/other lines are captured
chargingSplitMode	**PERCENT** · AMOUNT	Allocation entry mode
selfApprovalLimitSgd	int · **0**	Amount at/below which a traveller-approver self-approves (0 = never)
authorisationValidityDays / bookingDeadlineDays	int · **30 / 14**	TA validity & booking deadline (§13.3)
attachmentMaxMb	int · **7**	Max upload size per file
declarationText / coiText	text	Submission declarations

12 TMC integration

On approval (and hand-off), a **governed booking instruction** is assembled from the request, filtered by the admin Integration Contract (per-field include/treatment flags).

Hand-off payload (per instruction)

- **Authorisation** & traveller (or guest) identity; booking method; booking deadline; approval validity.
- **Air segments** — origin / destination (IATA, verbatim), depart date, approved class. **Hotel segments** — city, nights, capped nightly.
- Approved cost ceiling; approved exceptions; charging (primary bears the airfare posting); approval metadata.
- **Group fan-out (§9.3):** one approval → **one instruction per traveller**, each carrying that traveller's own class, nights and cost share.
- **Guest (§8):** a guest instruction carries `bookingFor = GUEST`, the guest identity, Economy class, and `settlement = HOST_DIRECT_BILL` with the host cost object.

> **Booking arrangement** decides the channel: AUTO routes to a TMC by policy/destination; a provider id targets a specific TMC; Self-booked / Host-arranged / No booking skip the hand-off. The resolved TMC is shown read-only on the trip form and the review/detail pages.

Integration Contract — field flags & message guards

The contract is a **declarative registry** (`src/config/integrationContracts.ts` , version `ecs-pretrip-contract/1.0` — stamped into every message's provenance header) governing the two integration boundaries: **Send to TMC** (the outbound booking instruction) and **Sync to TE** (the claim pre-population). Both `buildOutbound` and `prepopulateClaim` **read** it, so a toggle actually changes the payload — it is not a diagram. Defaults live in config (mapping to OutSystems site properties / exposed REST); admin overrides persist in `IntegrationFieldSetting` / `IntegrationGuardSetting` and merge at read time.

Send to TMC — outbound field flags

Core = always carried (not editable); **Toggle** = admin may include/exclude (all default **on**). Charging / cost objects are carried so airfare can post directly to ERP (§36); the CC/WBS itself is never used to *book*, only to post.

GROUP	FIELD	SOURCE	FLAG
Booking header	Travel Authorisation number	`req.authorisationNo`	**core**
Booking header	Traveller reference	`req.travellerId`	**core**
Booking header	Booking method / channel	`req.bookingMethod`	**core**
Booking header	All group travellers	`req.travellers[]`	toggle · on
Booking header	Booking deadline	`req.bookingDeadline`	toggle · on
Booking header	Approved cost ceiling (SGD)	Σ airfare + accommodation	toggle · on
Booking header	Approved policy exceptions	policyChecks(Exception)	toggle · on
Booking header	Travel purpose · Event dates · Approval validity	purpose / event dates / TA expiry	toggle · on
Itinerary	Air segments (origin / dest / date)	shared `legs[]`	toggle · on
Itinerary	Approved cabin class (ceiling)	`leg.travelClassId` → name	toggle · on
Accommodation	Hotel segments (city / nights / cap)	`accommodation[]`	toggle · on
Charging & context	Charging / cost objects (CC·WBS·BA)	`req.allocations[]`	toggle · on
Charging & context	Approval metadata (approvers + timestamps)	approvalSteps(Approved)	toggle · on
Charging & context	Traveller category & contact (or GUEST)	employee.policyGroup / guest	toggle · on
Charging & context	Personal-travel indicator	personalStart / personal legs	toggle · on

Sync to TE — claim pre-population treatment

Each field/line is **LOCKED** (copied, read-only) · **EDITABLE** (pre-filled, editable) · **REFERENCE** (shown for comparison) · **EXCLUDED** (not carried). Charging is LOCKED into the claim but **never sent to the TMC** (the opposite boundary).

GROUP	FIELD / LINE	DEFAULT TREATMENT
Claim header	Traveller · Entity · Department · TR number · TA number · Purpose · Description · Main destination	`LOCKED`
Claim header	Official travel dates	`EDITABLE` (trim to actual)
Claim header	Travel class (approved)	`REFERENCE`
Claim header	Charging & cost allocation	`LOCKED` — never to TMC
Claim header	Booking reference (PNR / ticket)	`REFERENCE` (§35, from the TMC feed)
Claim lines	Airfare	`REFERENCE` — overridden by the Airfare-treatment setting (§13.8)
Claim lines	Accommodation · ODA · Conference · Other	`EDITABLE`
Claim actuals	Booked amounts (fare / taxes / fees, TMC feed)	`REFERENCE` — set `EXCLUDED` for self-booked

Parameter filters (message guards)

Message-level gating: **all** of a flow's guards must hold before a message is sent/accepted. Each is a predicate (param · operator · value); an admin can enable/disable it and, where the operator is not mere presence, change the value. Enforced in `handoffToTmc` (OUTBOUND), `receiveBooking` (INBOUND) and claim creation (TE_SYNC).

FLOW	GUARD	TEST
OUTBOUND (send to TMC)	Request is approved	`status = Approved`
OUTBOUND	Not already handed off / booked	`bookingStatus = Not Sent`
OUTBOUND	Booking method is TMC-served	`bookingMethod ∈ {TMC_ONLINE, AGENT_ASSISTED, OFFLINE, HOST_ARRANGED}` (self-booked / no-booking never sent)
OUTBOUND	Travel Authorisation issued	`authorisationNo is set`
INBOUND (accept TMC response)	A booking instruction is in flight	`bookingStatus ∈ {Sent to TMC, Received by TMC, Booking In Progress, Traveller Action Required, Partially Booked}`
INBOUND	Travel Authorisation issued	`authorisationNo is set`
TE_SYNC (pre-populate claim)	Request is claimable	`status ∈ {Approved, Closed}`
TE_SYNC	Travel Authorisation issued	`authorisationNo is set`

A blocked message is **audited with the failing guard** (e.g. a hand-off attempted before approval, or a claim before a TA), so the admin sees exactly why a message did not flow. The contract version travels in the provenance header for payload-to-contract correlation (§13.9).

Inbound outcomes & failed-booking recovery (DD-15)

The OBT/TMC (e.g. **Cytric**, **Trip.Biz**) is a **fulfilment channel, not an approver**: it returns booking **status** — confirmed / issued / changed / cancelled / refunded — never an approval verdict. ECS normalises these into its own booking-status lifecycle (§4); "failed" is **inferred** from **not-issued within the booking window**, not assumed to be a single push event. A material booking deviation (fare over tolerance) raises a `BookingDeviation(action=Reapproval)` — again a deviation loop, not the OBT "sending the request back".

On **Booking Failed** the booking page shows a recovery panel, gated by the **booking window** (booking deadline / TA validity — a closed window means a re-send is unlikely to help):

RECOVERY	EFFECT
Re-send to TMC	retry the same instruction (fare/seat may have returned)
Amend & re-route	reopen to **Amendment In Progress**, re-derive the approval route, edit the itinerary — keeps a **single TA lineage**
Cancel & re-raise	the original goes terminal (Cancelled); a fresh request is then allowed under the duplicate guard (DD-14)

For an urgent trip the TMC typically books offline and contacts the traveller directly; once a replacement is booked, the original is cancelled so **no two authorisations stay live** — the duplicate guard (DD-14) blocks a re-raise until the original is cancelled or amended.

13 Reference masters (read from ECS)

IDENTITY

Employees & appointments

Department, business area, reporting officer, travel policy group, **defaultChargingCode** (Workday home-dept cost object), roles.

AUTHORITY

DOA lines & workflow bands

Per-department DOA line (+ research line); amount bands → DOA tier index + label.

FINANCE

Charging accounts

CC/WBS with company code, business area, department, research flag, funding owner, active/closed.

POLICY

Caps & ODA rates

Per-city hotel caps; per-country ODA daily rates; travel-class register (policy group, min hours, entitled class, effective dates).

LOCATION

Countries · cities · airports

IATA airport master (shared with the TMC); city → country; flight-hours from base.

RATES

FX & expense types

Entry-date FX (re-locked at submit); expense types with GL account & GST code.

VENDORS

TMC providers

Provider + region + booking methods (channels); policy routing.

14 Group & guest bookings

Group travel (DD-05) lets one request carry many travellers on a shared itinerary, each at their own class and cost. A group may also include **guest members** — non-employees with no ECS/HR profile (DD-13). Membership is capped at `groupMaxTravellers` (default **10**), the TMC group-fare limit.

Member model

Each member is a `TravelRequestTraveller` row. A guest is distinguished by `travellerType = GUEST` and a synthetic `employeeId` (a `G-…` key, since there is no HR id); its identity is captured inline.

FIELD	EMPLOYEE MEMBER	GUEST MEMBER
employeeId	ECS/HR id	synthetic `G-…` key
travellerType	EMPLOYEE	**GUEST**
guestName / guestEmail / guestOrg	—	captured on the create form
entitled / chosen class	derived per itinerary (DD-04)	**Economy** (no entitlement profile)
ODA	per-diem by destination	**none** — not a guest entitlement (blocked)
shared incidental	carries an equal share	**none** — never attributed a shared slice
charging	traveller's Workday department	**host (requestor) department**
settlement	own TE claim (reimbursed)	**TMC direct-billed to host** (no claim)
confirmation	each confirms inclusion (§13.13)	host confirms offline (pre-confirmed)

Creation & size limit

```
function createGroupDraft(employeeIds, guests, settings):   # DD-13
  total = employeeIds.length + guests.length
  if total < 2:                          reject  "a group needs ≥2 members"
  if guests and not settings.groupGuestMixAllowed: reject  "mixing disabled — use a guest re
  if total > settings.groupMaxTravellers: reject  "over the TMC group limit"
  for eid in employeeIds: assert canCreateFor(requestor, eid)   # §13.13 delegation
  members = employeeIds.map(EMPLOYEE) + guests.map(g => GUEST("G-"+uid(), g))
  share = 100 / total                    # equal share; roll-up drives routing
  create TravelRequest(isGroup, travellers = members)
```

The create form offers the full traveller pool plus inline guest rows (name / email / organisation), shows a live **"N of max M"** counter, and blocks submit when over the limit or under two members. **Add-traveller** on an existing group re-checks the same cap.

Bulk member import (Excel / CSV)

For a large party, the coordinator can upload a member list instead of ticking each traveller. The file is parsed **client-side** (SheetJS) and merged into the current selection — it is a convenience over the same selection state, not a separate submission path. Accepted: `.xlsx` · `.xls` · `.csv`. Columns are matched case-insensitively.

COLUMN	MEANING
Type	`Employee` | `Guest` — optional; blank rows auto-classify (see below)
Employee ID	ECS id for an employee member (e.g. `E-TRAV3`)
Name	Employee name (alternative to id) or guest name
Email	Guest email, or an employee email used to match
Organisation	Guest's home organisation

```
for row in sheet:                                    # per uploaded row
  if type == "guest":                              addGuest(name, email, org)
  elif type == "employee" or employeeId:            # employee intent
     emp = resolve(employeeId or name or email)     # by id → email → name
     emp ? selectEmployee(emp) : skip("no match: "+key)
  else:                                             # untyped row
     emp = resolve(name or email)
     emp ? selectEmployee(emp) : addGuest(name, email, org)
```

- **Resolution:** employees match the ECS directory by id, then email, then name; an unmatched employee row is reported as a **skipped row with its reason** (nothing

invented).

- **Merge & dedup:** imported members merge into the current selection — employees deduped by id, guests by name — and the running total still honours `groupMaxTravellers` via the live counter.
- **Editable after import:** every imported member lands as a normal checkbox / guest row the coordinator can adjust or remove; upload and manual entry mix freely.
- **Template:** a "Download template" action emits a prefilled workbook — a *Members* sheet (one employee + one guest example) and an *Employee directory* reference sheet of valid ids.

> The upload never bypasses governance: it only fills the **same** selection the manual UI does, so the size limit, delegation check (§13.13) and per-member guest/Economy rules all still apply at submit. No server round-trip is needed to parse the file.

Configurable mixing (`groupGuestMixAllowed` , default on). When off, a group is employees-only: `createGroupDraft` rejects any guest, the create form replaces the guest section with a pointer to the standalone guest request, and the Excel import skips guest rows with a reason.

Downstream handling

- **Entitlement (§8):** a guest short-circuits to Economy; employees derive per itinerary. Each member's class is stamped on their own airfare line.
- **Charging (§9):** a guest's cost maps to the host department's cost object; the per-department roll-up still derives the DOA from the highest-share department. Guest cost is never split onto the guest.
- **No shared slice, no ODA:** shared incidentals split across **employee members only**, and ODA is **blocked for guests at authoring**. Both are **validated at submit** — a shared line that names a guest, or a guest ODA line, is a hard block. (A guest's only settlement path is direct-billing, which covers neither.)
- **Review & detail:** the roster shows each member with a **Guest** pill, their org (*Guest · MIT*), and Economy class.
- **TMC fan-out (§12):** a guest instruction carries `bookingFor = GUEST` , the guest identity block, Economy class, and `settlement = HOST_DIRECT_BILL` with the host cost object.
- **Claim:** claims are keyed to the logged-in persona, so a guest — having no login — can never produce one; the host absorbs guest cost.

Guest settlement — host direct-billed

Because a guest files no claim, their cost is **not** reconciled through the claim pipeline. It is host-settled: the TMC **direct-bills the host department cost object**, and the actual is read from the guest's own booking in the group fan-out. The group detail page carries a **Guest Settlement** card:

COLUMN	SOURCE
Cost object (host)	host (requestor) department default, or an explicitly-charged guest line (§9 edit)
Estimate (SGD)	the guest's own **airfare + accommodation** only (the direct-billable categories)
Direct-billed (SGD)	the guest's booking actual (fare + taxes + fees + hotel), once booked
Settlement	`Pending TMC booking` → `Direct-billed to host` after the fan-out confirms

A per-person upgrade, or a guest who must claim their own cost, is handled by **removing them from the group and raising a single request** (the DD-05 escape hatch), not by special-casing the group.

15 Enumerations — allowed values

SQLite has no native enums, so status / type fields are constrained strings. The allowed value sets below are the single source of truth — a field must never hold a value outside its set. **Defined value sets** live in `src/shared/enums.ts`; **inline string-enums** are documented in the schema. A companion **field-catalogue workbook** (`docs/ecs-pretrip-field-catalogue.xlsx`) carries the same sets per field.

Defined value sets (enums.ts)

ENUM	USED BY	ALLOWED VALUES
REQUEST_STATUS	`TravelRequest.status`	`Draft` `Submitted` `Pending RO Approval` `Pending Additional Approval` `Pending Exception Approval` `Pending DOA Approval` `Approved` `Amendment In Progress` `Sent Back` `Withdrawn` `Cancelled` `Expired` `Closed` · `Rejected` (legacy/compat, not a transition target — DD-12)
BOOKING_STATUS	`TravelRequest.bookingStatus`	`Not Sent` `Sent to TMC` `Received by TMC` `Booking In Progress` `Traveller Action Required` `Partially Booked` `Booked` `Travel Completed` `Booking Failed` `Self-Booked` `Cancelled`
POLICY_OUTCOME	`PolicyCheck.outcome`	`PASS` `WARNING` `EXCEPTION` `HARD_STOP` **`HARD_STOP blocks submission`**
APPROVER_ROLE	`ApprovalStep.roleType` · `ClaimApprovalStep.roleType`	`RO` `ADDITIONAL_APPROVER` `DOA` `RESEARCH_DOA` `EXCEPTION` `FUNDING_OWNER` `VERIFIER` (claim stage only)
CLAIM_STATUS	`TravelExpenseClaim.status`	`Draft` `Pending Verification` `Pending Exception Approval` `Pending RO Approval` `Pending DOA Approval` `Approved` `Sent Back`
TE_TREATMENT	`TEExpenseLine.treatment` · `IntegrationFieldSetting.teTreatment`	`LOCKED` `EDITABLE` `REFERENCE` `EXCLUDED`
BOOKING_METHOD	`TravelRequest.bookingMethod`	`TMC_ONLINE` `AGENT_ASSISTED` `OFFLINE` `HOST_ARRANGED` `SELF_BOOKED` `NO_BOOKING`
BOOKING_ARRANGEMENT	`TravelRequest.bookingArrangement`	`AUTO` `TMC-<id>` `SELF_BOOKED` `HOST_ARRANGED` `NO_BOOKING`
EXPENSE_CATEGORY	`EstimatedExpense.category` · `TEExpenseLine.category`	`AIRFARE` `ACCOMMODATION` `ODA` `CONFERENCE` `OTHER`
ROLE	`Employee.roles` (ECS persona master)	`TRAVELLER` `TRAVEL_REQUESTOR` `RO` `DOA` `RESEARCH_DOA` `EXCEPTION_APPROVER` `TRAVEL_ADMIN` `SYSTEM_ADMIN` `FINANCE_VIEWER`

Inline string-enums (schema)

FIELD	ALLOWED VALUES
TravelRequest.tripType	`ROUND_TRIP` `ONE_WAY` `MULTI_CITY`
travellerType	`EMPLOYEE` `GUEST` (non-employee, no ECS/HR profile — §8)
EstimatedExpense.estimateBasis	`QUOTED` `CAP` `LOWER RATE`
ChargingAllocation.chargingType	`CC` `WBS`
TravelRequest.chargingMode	`REQUEST` `LINE`
TravelExpenseClaim.chargingMode	`MAIN` `CLAIM ITEM`
ItineraryLeg.transportMode	`AIR` `RAIL` `OTHER`
BookingSegment.type	`AIR` `HOTEL`
TravelBooking.channel	`TMC` `SELF_BOOKED`
BookingDeviation.action	`Reapproval` `LogOnly` `UpdateBooking`
TEExpenseLine.receiptType	`LOCAL_GST` `FOREIGN_NO_GST` `NO_RECEIPT`
AccommodationEstimate.exceptionOutcome	`PASS` `EXCEPTION`
IntegrationMessage.direction / kind	`OUTBOUND` `INBOUND` / `TMC_HANDOFF` `TMC_RESPONSE`
AuditEvent.kind	`CREATE` `SUBMIT` `APPROVE` `REJECT` `SENDBACK` `AMEND` `INTEGRATION` `TE_LINK` `STATUS` `COMMENT`
Attachment.docType	`Quotation` `Conference` `Invitation` `Approval` `Evidence` `Other`

Module Settings — config enums

SETTING	ALLOWED VALUES (DEFAULT IN BOLD)
expenseScope	MAJOR_ONLY **ALL**
hotelEstimateBasis	QUOTED CAP **LOWER**
airfareTreatment	DIRECT_SAP **IN_TE** REFERENCE
teLinkageMandatory	YES NO **CONDITIONAL**
approvalAmountBasis	GROSS **NET_NTU** MAJOR_COST ALLOCATION
roRequirement	ALWAYS **INDIVIDUAL_ONLY** NEVER
chargingSplitMode	**PERCENT** AMOUNT

The approval- and booking-status **transition rules** (which value may follow which) are in §4; this section lists the value **domains**. The boolean / numeric Module Settings are in §11.

16 Open design decisions

Decisions still to be made — **not yet built**. Each states the question, the current behaviour, and a lean; the lean is a starting point for discussion, not a commitment. Numbered `OD-nn` so they can be promoted to a `DD` once settled.

`P1 · decide before build` `P2 · decide early` `P3 · can defer`

P1 is architectural or a release blocker (shapes the data model or integration contract); **P2** shapes the design but has a workable default today; **P3** is a refinement or edge case that can follow.

`OD-01`

Should a TE claim (and its booking) support multiple PNRs? `P1 · decide before build`

Question: A single trip can carry more than one PNR — a multi-city itinerary split across carriers (e.g. SIN→LON on PNR A, LON→PAR→SIN on PNR B), a separate hotel PNR, or a re-issue after a change. Should one request/claim reconcile **N** PNRs?

Today: a `TravelBooking` holds one PNR/ticket; the group fan-out creates one booking (one PNR) *per traveller*, but a single traveller's multi-city trip is modelled as one booking → one PNR. The TE claim reconciles to that single booking's actuals.

To decide: (a) allow **many bookings (PNRs) per request**, each with its own ticket/segments; (b) **map each PNR to its legs** so a cost line reconciles to the right ticket; (c) run the booked-vs-approved deviation check **per PNR or on the aggregate**; (d) how the claim displays and totals multiple PNRs. **Lean:** one booking record per PNR under the request, PNR↔leg mapping, deviation on the aggregate airfare, claim aggregates across PNRs.

`OD-02`

Emergency / retrospective booking ratification lane `P2 · decide early`

Question: When urgency leaves no time for a pre-trip approval cycle, should there be a lane where the TMC books offline and the request is **ratified after the fact** within policy, rather than bending amend/re-raise to the clock?

Today: every booking requires an approved TA first; there is no retrospective path. **Lean:** add a bounded emergency lane (offline booking → retrospective ratification within tolerance, else full re-approval).

OD-03

Out-of-policy / over-ceiling at booking: rebook vs re-approve `P2 · decide early`

Question: When the only available fare is out of policy or over the approved ceiling, should the request **return to the traveller to rebook within policy**, or route **straight to re-approval of the higher amount**? (The real decision hiding inside "can the OBT send it back".)

Today: a material fare deviation raises `BookingDeviation(action=Reapproval)`. **Lean:** small/explained overages → re-approve; large/avoidable → return to rebook; make the split a tolerance/policy setting.

OD-04

Guest honorarium / per-diem — in scope or separate? `P3 · can defer`

Question: A guest gets no ODA here (DD-16). If a guest is nonetheless due a per-diem or honorarium, is that **out of scope** for Pre-Trip (handled as a separate direct payment / AP), or should the module capture it as a distinct, non-claim line?

Today: guest cost is airfare + accommodation only (host direct-billed). **Lean:** keep honorarium out of Pre-Trip; if needed, capture as a separate host-settled line, never as ODA or a claim.

OD-05

Finalise the OBT inbound event mapping (Cytric / Trip.Biz) `P1 · at integration`

Question: The normalized inbound status set (§12) must be mapped to each OBT's real callbacks. Trip.Biz publishes `paid / issued / cancelled / refunded / changed`; Cytric's exact events are partner-gated. Which events map to **Booked**, **Failed (not-issued)**, **Traveller Action Required**, etc., and is failure **pushed or inferred** per vendor?

Today: ECS normalizes to its own booking-status lifecycle; "failed" is inferred from not-issued within the window. **Lean:** finalise the per-vendor event map at integration onboarding; keep the normalized set as the contract.
